

Inference

- Given: observed sentence $O = [o_1, \dots, o_T], o_t \in V$
a fixed HMM with parameters (A, B, π)
- Find: $\Pr(O|A, B, \pi)$
- Useful for evaluating the validity of a sentence.
- By total probability, $\Pr(O|A, B, \pi) = \sum_Q \Pr(O, Q|A, B, \pi)$

- The summation is over all possible hidden state sequences

$$Q = [q_1, \dots, q_T], q_t \in S$$

- A brute-force enumeration will have time complexity $O(|S|^T)$

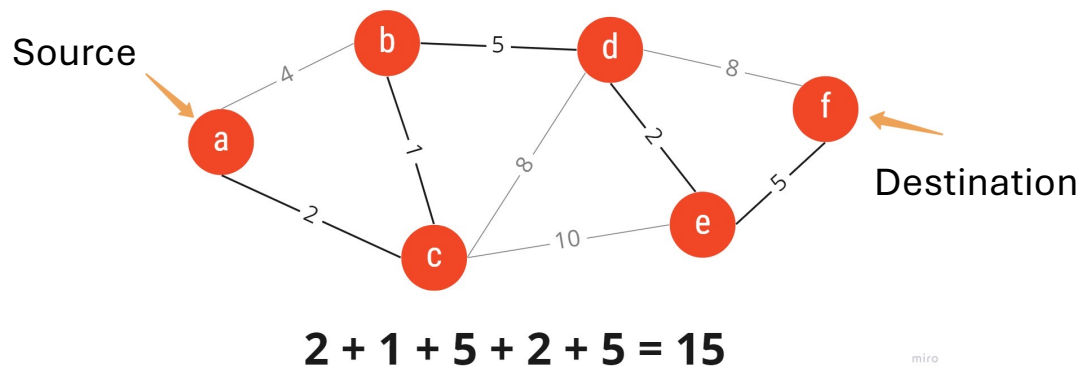
a_{ij} : (i, j) entry of A, transition probability from tag i to j.

b_{jo} : (j, o) entry of B, emission probability of token o from tag j.

π_i : entry i of π , probability of starting from tag i.

Forward algorithm

- Forward algorithm: find $\Pr(O|A, B, \pi) = \sum_Q \Pr(O, Q|A, B, \pi)$ without brute force.
 - It is an instance of dynamic programming algorithm:
 - solves a large problem using **cached solutions** to sub-problems
 - **overlapping sub-problems**: can re-use solutions to sub-problems;
 - **optimal structures**: optimal sub-problem solutions => optimal larger problem solutions.



- Re-use solution (a-c-b) to construct the solution (a-c-b-d), without solving shortest path from a to b again.
- If (a-c-b-d) is the shortest path from a to d, then (a-c-b) must be the shortest path from a to b.

Forward algorithm

- Two simple identities:

- Product and summation operations can be exchanged over real numbers

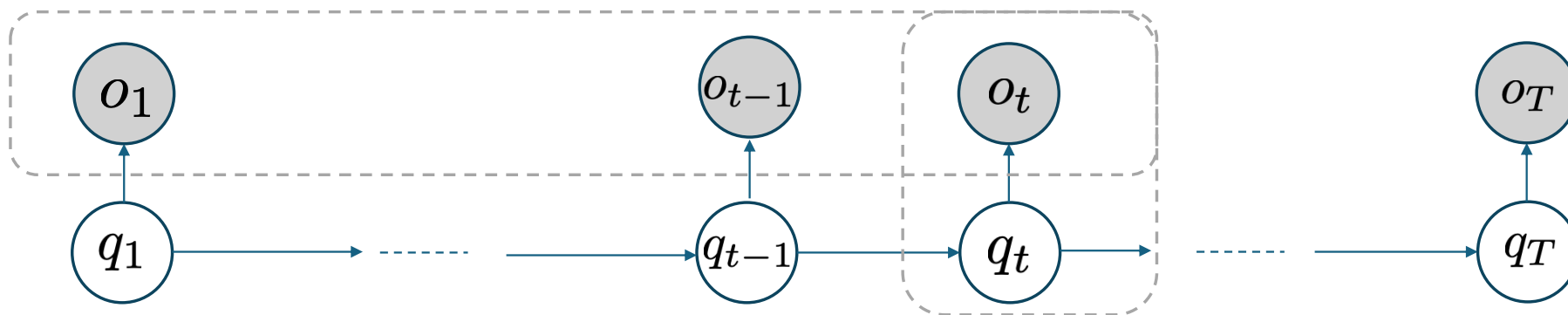
$$ax + ay + bx + by = (a + b)(x + y)$$

- Product and max operations can be exchanged over positive numbers

$$\max(ax, ay, bx, by) = \max(a, b) \times \max(x, y)$$

- Define the *forward* probability $\alpha_t(j) = P(o_1, \dots, o_t, q_t = j)$

History tracer:
recall the past



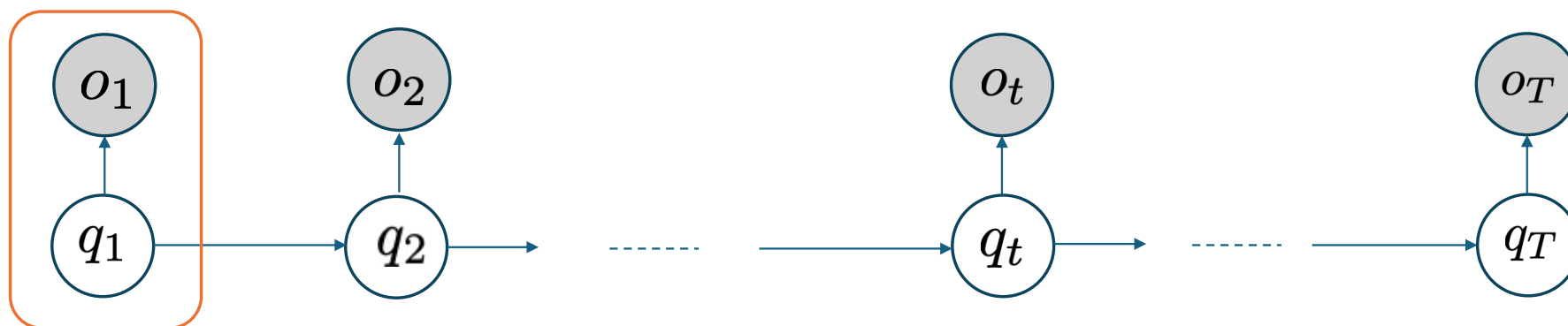
Forward algorithm

- A gentle start: compute the special case

$\alpha_1(i) = \pi_i \times b_i(o_1)$ Joint probability of first tag being i and seeing o_1

π_i : the probability of the first hidden state (POS tag) being i .

$b_i(o_1)$: the emission probability of the first observed word o_1 given state i .



Forward algorithm

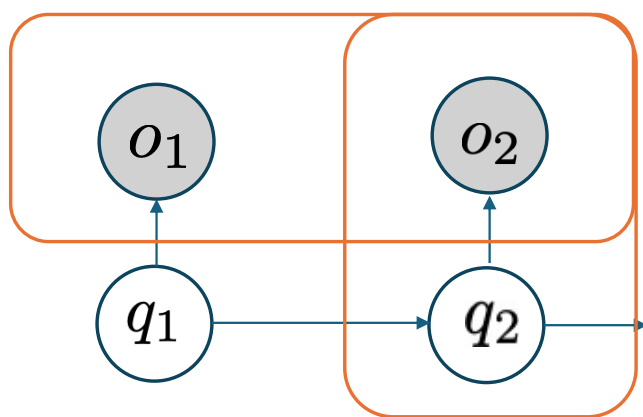
Note: we let $N = |S|$

- One step further: use to $\alpha_1(i)$ compute $\alpha_2(j)$

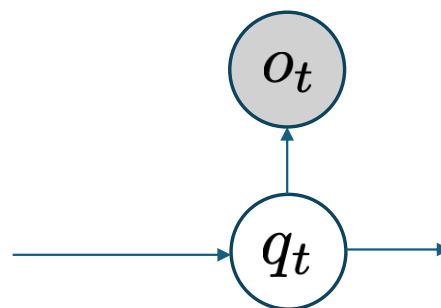
$\alpha_2(j)$: the probability of q_2 being in state j and observing tokens up to step 2.

- Account for **all possible paths (of length 2)** that could have led there from the previous time step

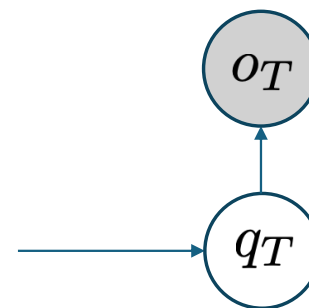
- **Marginalize** previous state q_1 as a random variable.
- **Reuse** previously calculated probability $\alpha_1(i)$ (solution of sub-problems).



.....



.....

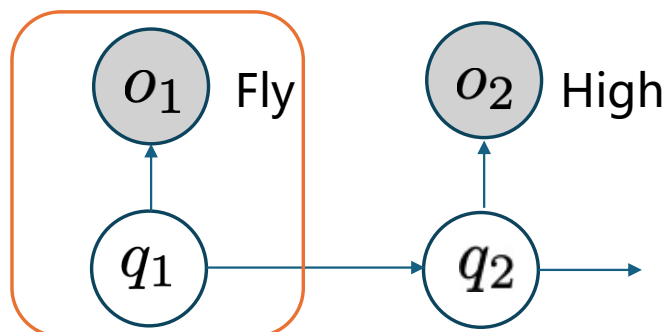


$$\alpha_2(j) = \left[\sum_{i=1}^N \alpha_1(i) \times a_{ij} \right] \times b_j(o_2)$$

transition prob. emission prob.

Forward algorithm example

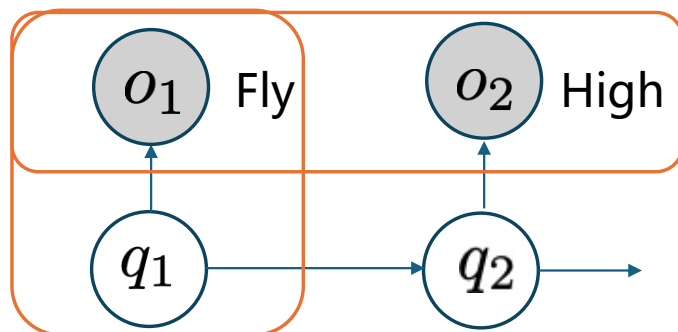
Step 1



$$\alpha_1(N) = \pi_N b_N(\text{fly})$$

$$\alpha_1(V) = \pi_V b_V(\text{fly})$$

Step 2



$$\alpha_2(N) = \sum_{k \in \{N, V\}} \alpha_1(k) \times a_{\{k \rightarrow N\}} b_N(\text{high})$$

$$\alpha_2(V) = \sum_{k \in \{N, V\}} \alpha_1(k) \times a_{\{k \rightarrow V\}} b_V(\text{high})$$

Forward algorithm

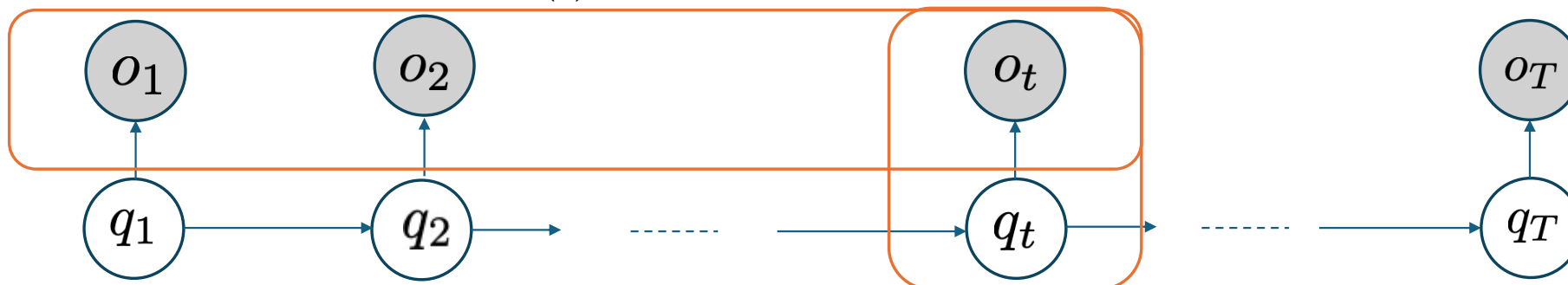
Note: we let $N = |S|$

- The general case

$$\alpha_t(j) = \sum_{k=1}^N \alpha_{t-1}(k) \times a_{kj} \times b_j(o_t)$$

- Proof:

$$\begin{aligned} P(o_{1:t}, q_t = j) &= \sum_{k=1}^N P(o_{1:t}, q_{t-1} = k, q_t = j) = \sum_{k=1}^N P(o_{1:t-1}, q_{t-1} = k) \times P(q_t = j, o_t | o_{1:t-1}, q_{t-1} = k) \\ &= \sum_{k=1}^N P(o_{1:t-1}, q_{t-1} = k) \times P(q_t = j | q_{t-1} = k) \times P(o_t | q_t = j) \\ &= \sum_{k=1}^N \underbrace{P(o_{1:t-1}, q_{t-1} = k)}_{\alpha_{t-1}(k)} \times a_{kj} \times b_j(o_t) \end{aligned}$$



Forward algorithm

Dynamic programming at work!

- Just curious, if we recursively plug in previous forward probabilities and expand them, what do you get?

Time complexity

$$O(2N) \quad \alpha_t(j) = \left[\sum_{q_{t-1}=1}^N \left(\sum_{q_{t-2}=1}^N \alpha_{t-2}(q_{t-2}) a_{q_{t-2}, q_{t-1}} b_{q_{t-1}}(o_{t-1}) \right) a_{q_{t-1}, j} \right] b_j(o_t)$$

$$O(N^2) \quad \alpha_t(j) = \sum_{q_{t-1}=1}^N \sum_{q_{t-2}=1}^N \alpha_{t-2}(q_{t-2}) \cdot [a_{q_{t-2}, q_{t-1}} b_{q_{t-1}}(o_{t-1})] \cdot [a_{q_{t-1}, j} b_j(o_t)]$$

$$O(N^t) \quad \alpha_t(q_t) = \sum_{q_1=1}^N \sum_{q_2=1}^N \cdots \sum_{q_{t-1}=1}^N \pi_{q_1} b_{q_1}(o_1) \prod_{i=2}^t a_{q_{i-1}, q_i} b_{q_i}(o_i)$$

Forward algorithm

- The probability of the sentence O

$$\Pr(O) = \sum_j P(O, q_T = j) = \sum_j \alpha_T(j)$$

- Eliminate the two inner loops

$$\alpha_t(j) = \underbrace{\alpha_{t-1}}_{\alpha_{t-1}(k)} \times A \otimes b_t$$

Forward algorithm: compute the likelihood of an entire sequence

1. Initialize $\alpha_1(i)$ for each value i of q_1 .
2. for $t = 2, \dots, T$
for $j = 1, \dots, N$
compute $\alpha_t(j) = \sum_k \alpha_{t-1}(k) a_{kj} b_j(o_t)$
3. Return $\Pr(O) = \sum_j P(O, q_T = j) = \sum_j \alpha_T(j)$

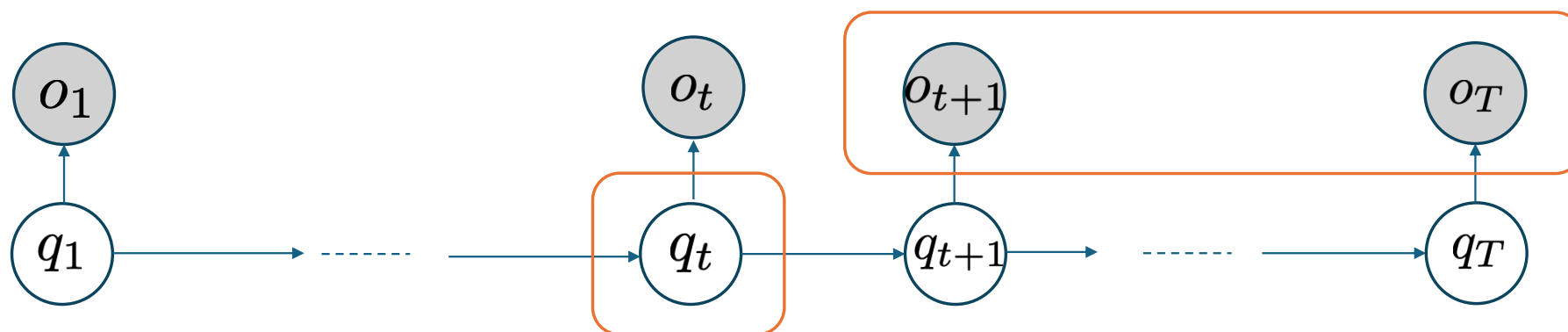
Backward algorithm

- Using the same principles (DP, marginalization), we can calculate the marginalization

$$\Pr(O|A, B, \pi) = \sum_Q \Pr(O, Q|A, B, \pi)$$

- Define the *forward* probability $\beta_t(j) = P(o_{t+1}, \dots, o_T | q_t = j)$

Fortune Teller:
predict the future



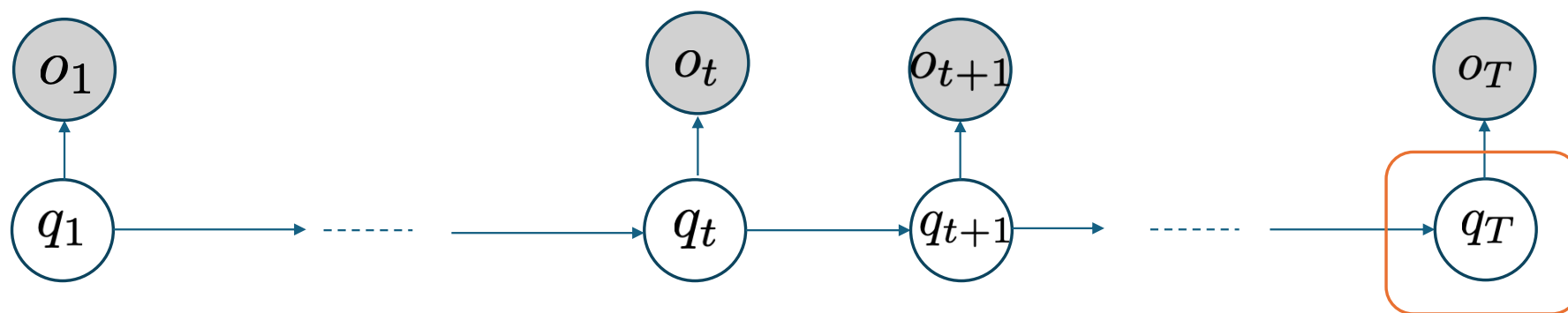
Backward algorithm

- Starting from the end of the sentence.

$$\beta_T(i) = 1$$

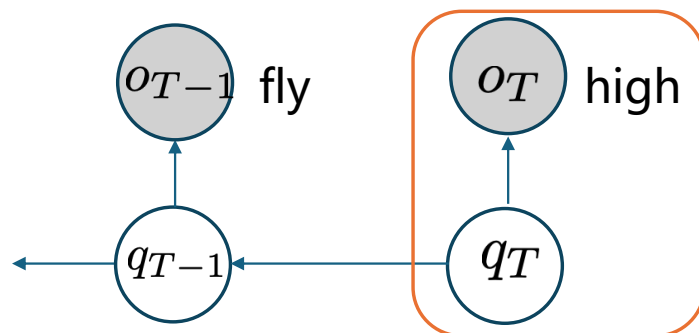
This initialization represents the probability of an "empty" future sequence. Mathematically, it acts as the neutral element (1) for the products that will be accumulated as we move backward.

$$\beta_{T-1}(i) = \sum_{j=1}^N a_{ij} \times b_j(o_T) \times \beta_T(j)$$



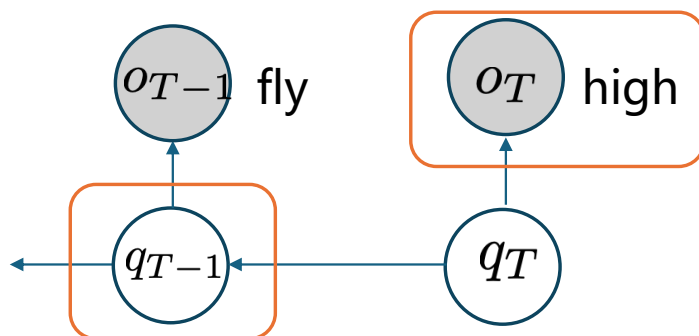
Backward algorithm example

Step 1



$$\beta_T(N) = \beta_T(V) = 1$$

Step 2



$$\beta_{T-1}(N) = \sum_{k \in \{N, V\}} a_{\{N \rightarrow k\}} \times \beta_T(k) b_k(\text{high})$$

$$\beta_{T-1}(V) = \sum_{k \in \{N, V\}} a_{\{V \rightarrow k\}} \times \beta_T(k) b_k(\text{high})$$

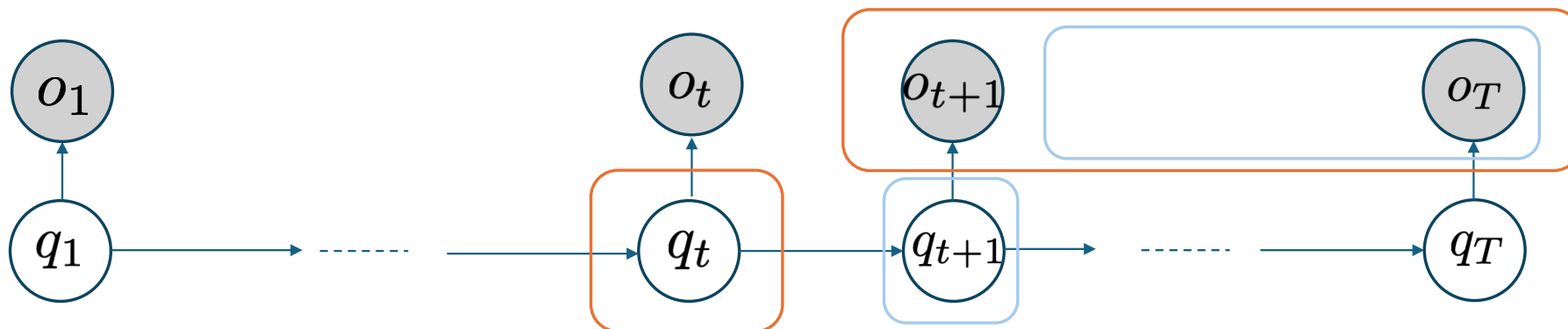
Backward algorithm

- The general case

$$\beta_t(i) = \sum_{j=1}^N a_{ij} \times b_j(o_{t+1}) \times \beta_{t+1}(j)$$

- Proof:

$$\begin{aligned} P(o_{t+1:T}|q_t = i) &= \sum_{j=1}^N P(o_{t+1:T}, q_{t+1} = j | q_t = i) = \sum_{j=1}^N P(q_{t+1} = j, o_{t+1} | q_t = i) \times P(o_{t+2:T} | q_{t+1} = j, o_{t+1}, q_t = i) \\ &= \sum_{j=1}^N \underbrace{P(q_{t+1} = j | q_t = i)}_{a_{ij}} \times \underbrace{P(o_{t+1} | q_{t+1} = j)}_{b_j(o_{t+1})} \times \underbrace{P(o_{t+2:T} | q_{t+1} = j)}_{\beta_{t+1}(j)} \end{aligned}$$



Forward algorithm

- The probability of the sentence O

$$\Pr(O) = \sum_{j=1}^N \pi_j \times b_j(o_1) \times \beta_1(j)$$

- Eliminate the two inner loops

$$\beta_t(j) \left\{ \beta_t \right\} = \underbrace{\beta_{t+1}}_{\beta_{t+1}(k)} \times A^T \otimes b_t$$

Backward algorithm: compute the likelihood of an entire sequence

Initialize $\beta_T(i)$ for each value i of the last hidden state q_T .

for $t = T - 1, \dots, 1$

for $j = 1, \dots, N$

compute $\beta_t(j) = \sum_k \beta_{t+1}(k) a_{jk} b_j(o_t)$

Return $\Pr(O) = \sum_j P(q_1 = j) b_j(o_1) \beta_1(j)$

Extra readings

- For forward and backward algorithm, see Chap 2 of SLP3.
- For dynamic programming, see https://en.wikipedia.org/wiki/Dynamic_programming
- For probabilistic graphical models, see the big book “[Probabilistic Graphical Models](#)” by D Koller.
 - Calculating marginal probabilities (called “inference”) is a central task on PGM, see Chap 9-10. In particular, check the “message passing” algorithm.
 - Finding the parameters (such as A, B, π for HMM) is called “learning” on PGM.

| Demon

- Using the texts with POS tags we generated from the previous demo, now let's construct an HMM model and do some inference.
- Fix the ground truth parameters.
- Run the forward algorithm.
- Run the backward algorithm.

| Conclusion

- Forward and backward algorithms are two algorithms for calculating marginal distribution of observed sentences using HMM.
- Both are dynamic programming algorithms.
- Next: Learning the HMM model.
- Let us know your thoughts using the QR code on Canvas

Quiz

1. Pen and paper; no compute or cellphone allowed.
 2. Turn in your answer sheet when you leave the classroom.
-
- Q1: for each time step of the forward algorithm, what is the time complexity given N POS tags?
 - A1: $O(N^2)$
 - Q2 (T or F): both forward and backward algorithms use dynamic programming principles.
 - A2: T
 - Q3 (T or F): the backward probability at the end of a sentence is always 1.
 - A3: T